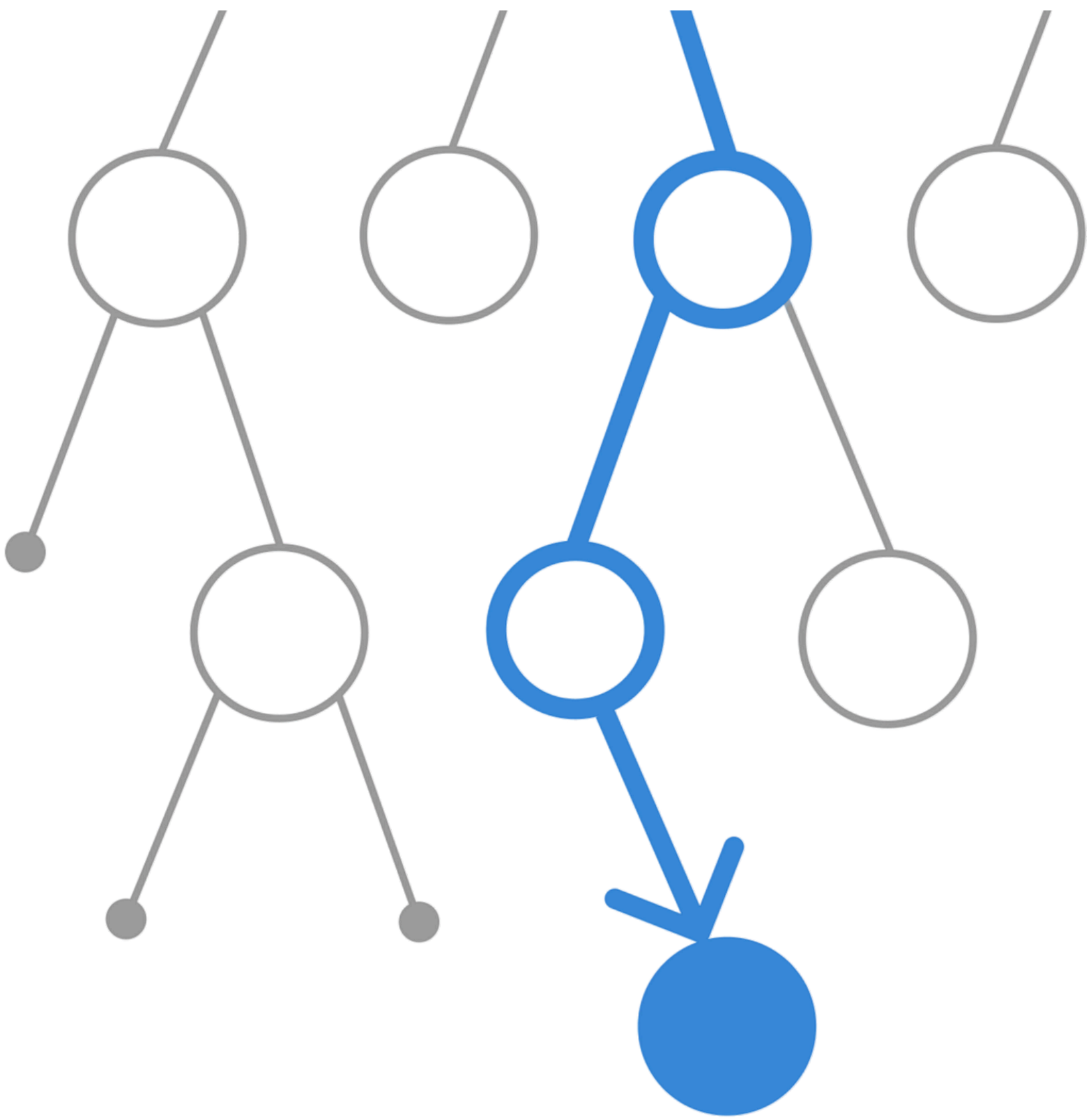
PageIndex vs Traditional RAG

RAG doesn’t fail because of bad embeddings. It fails because it searches instead of navigating.



RAG is broken for document chatbots.

Not because of bad models.
Not because of bad data.

Because the approach itself is flawed.

If you've built a RAG system, you've seen this:

- "The answer is in the doc... but it can't find it"
- "It keeps pulling the wrong chunk"
- "Works in demo, fails in production"

Sound familiar?

What RAG Actually Does

RAG pipeline in one line:

Chunk → Embed → Store → Retrieve → Generate

Simple. Popular. Everywhere.

But here's the problem...

Problem #1

Chunking destroys context

- Tables get split
- Sections break apart
- Important info lives across chunks

Documents lose their meaning.

Problem #2

Similarity ≠ Relevance

You ask: "termination clause"

Document says: "Section 14.3 – Dissolution"

RAG misses it. Because it matches wording, not meaning.

The Real Insight

Here's the thing:

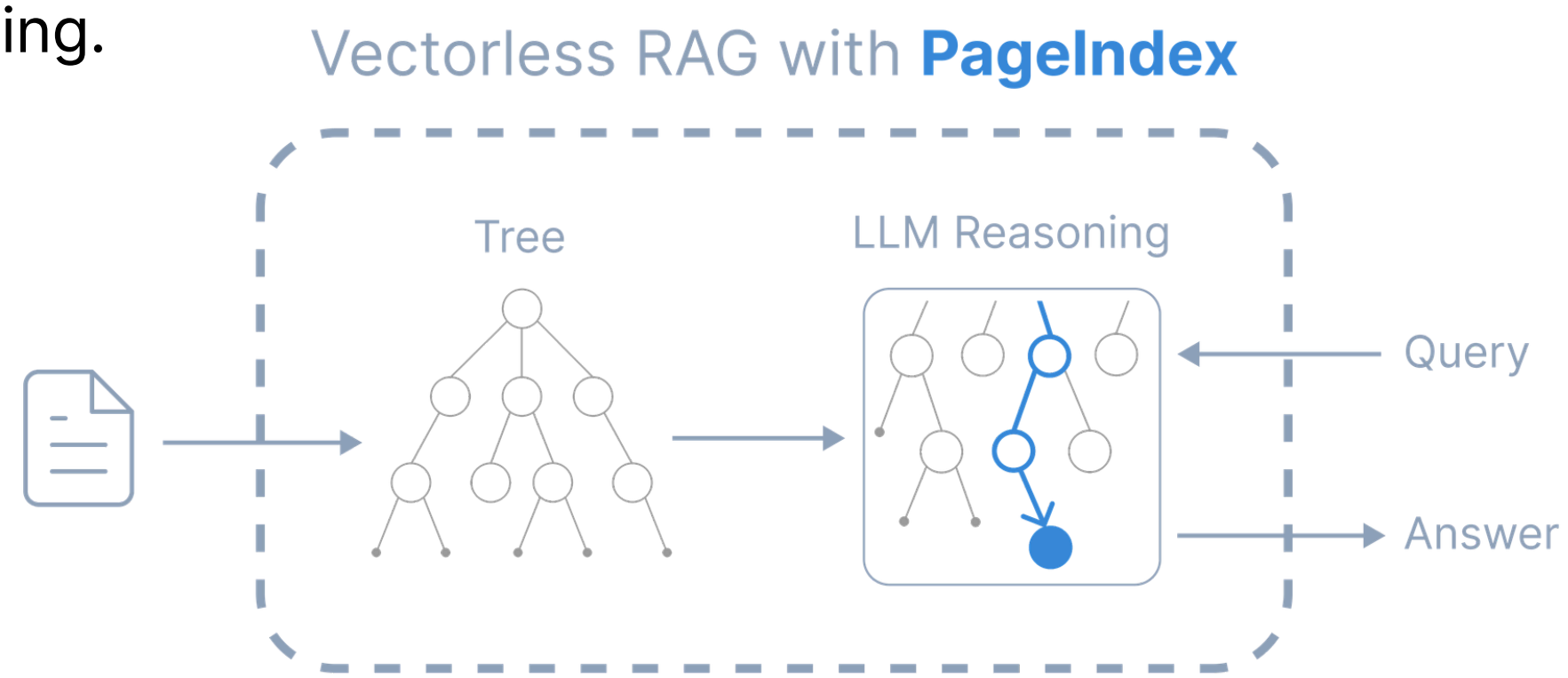
RAG treats documents like text. But documents are structured.
Humans don't search documents. We navigate them.

Enter PageIndex

A completely different approach:

- No chunking
- No embeddings
- No vector database

Just reasoning.



How PageIndex Works

Think of it as a smart table of contents:

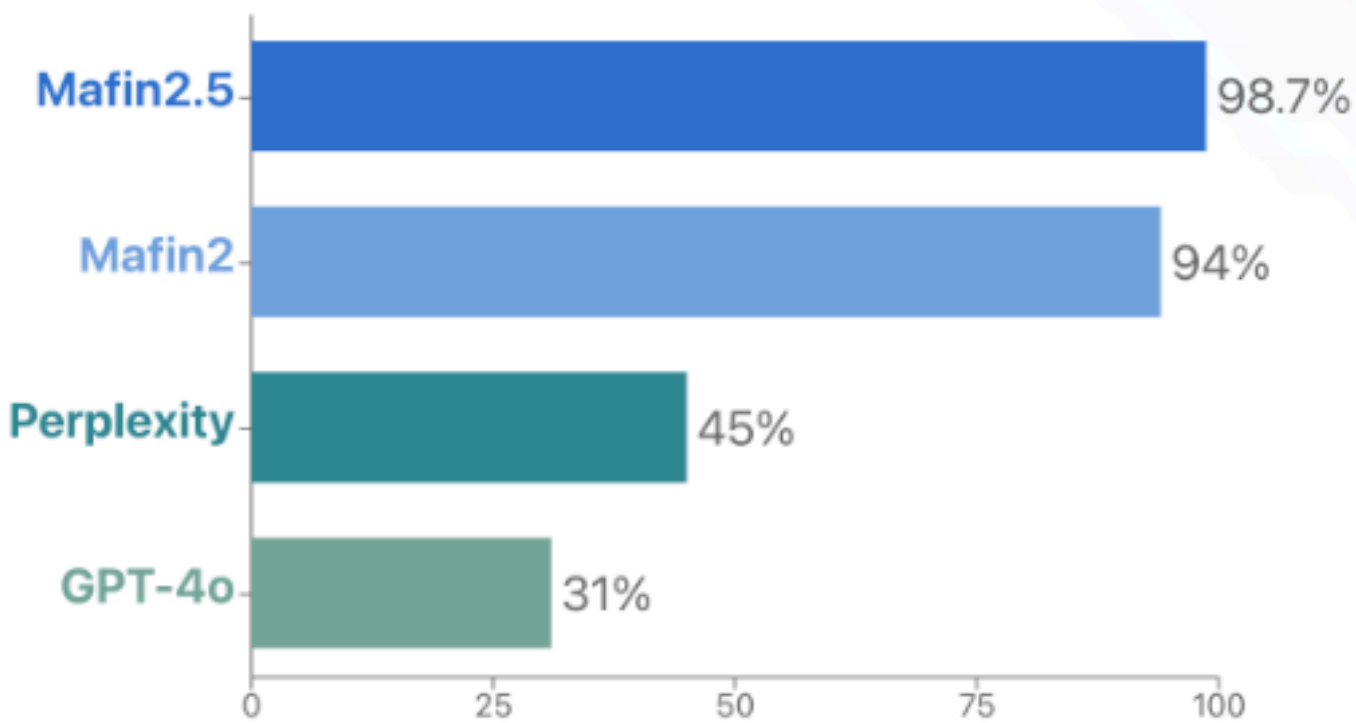
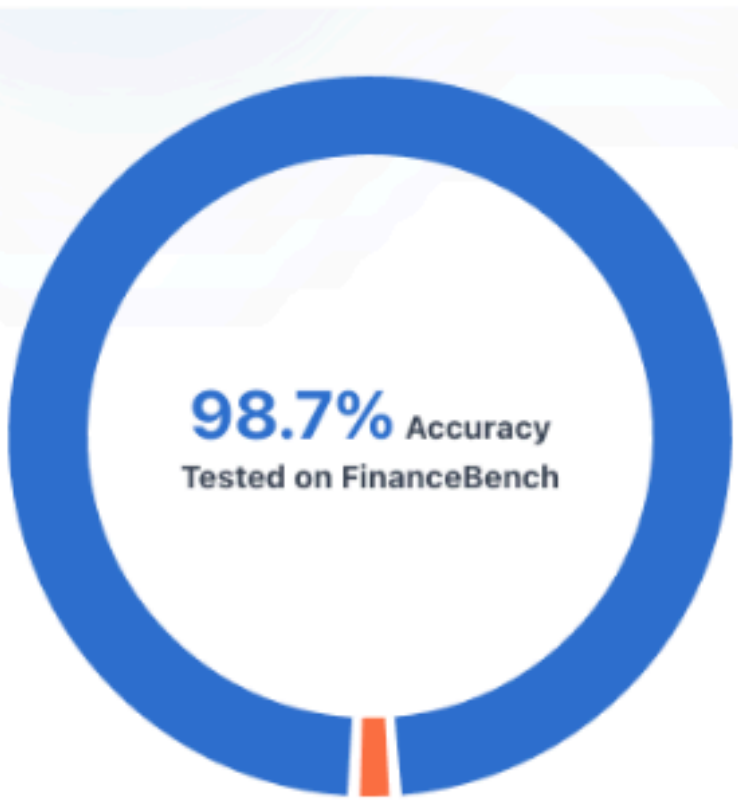
1. Builds a **Reasoning Tree**
2. **LLM navigates** to the **right section**
3. **Generates answer** from that section

Why It Wins

PageIndex fixes what RAG breaks:

- Preserves document structure
- Finds the right section (not similar text)
- Handles semantic mismatch
- Fully explainable answers

Proof

98.7% accuracy on FinanceBench.

One of the hardest benchmarks for document QA.